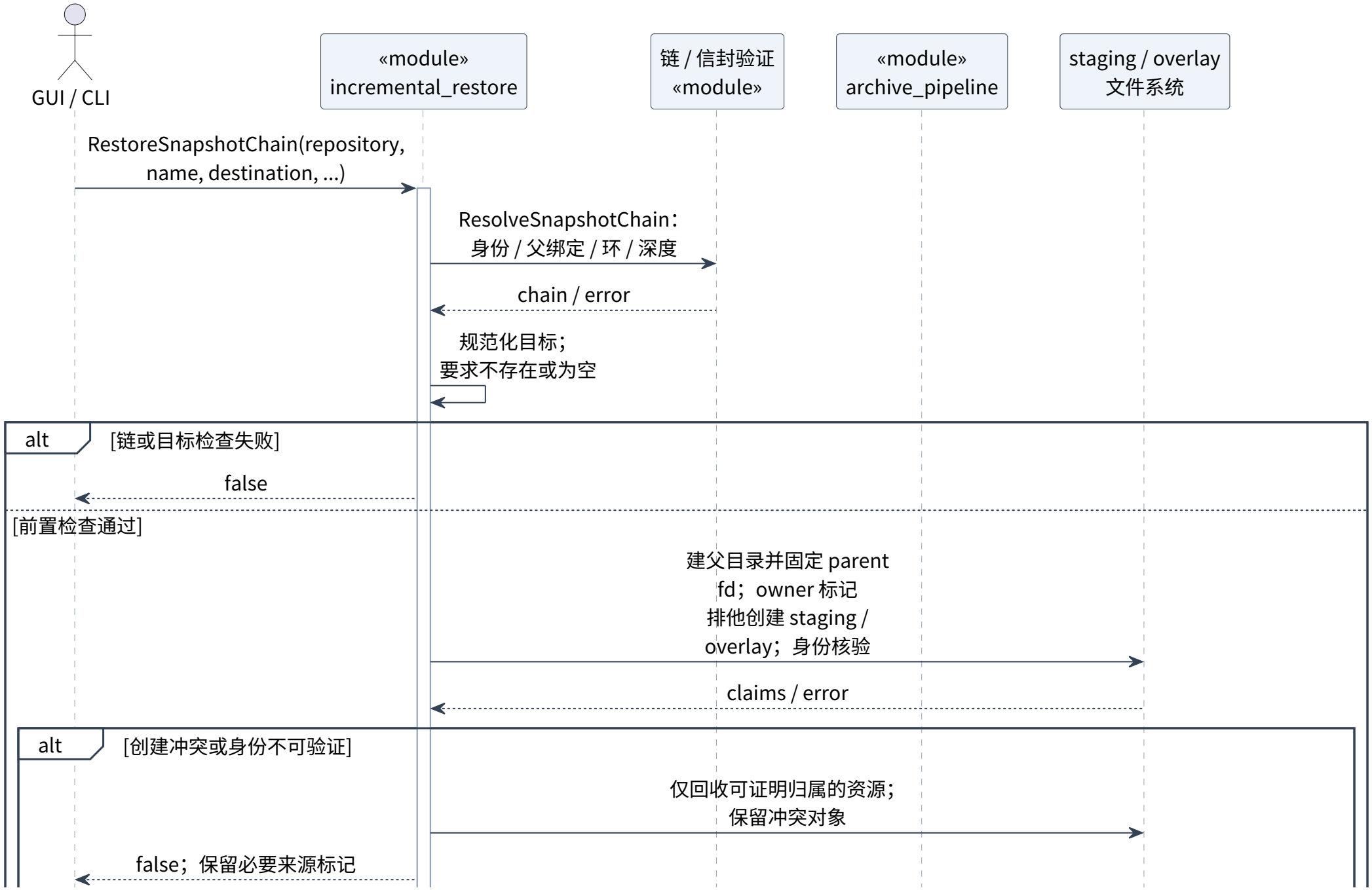
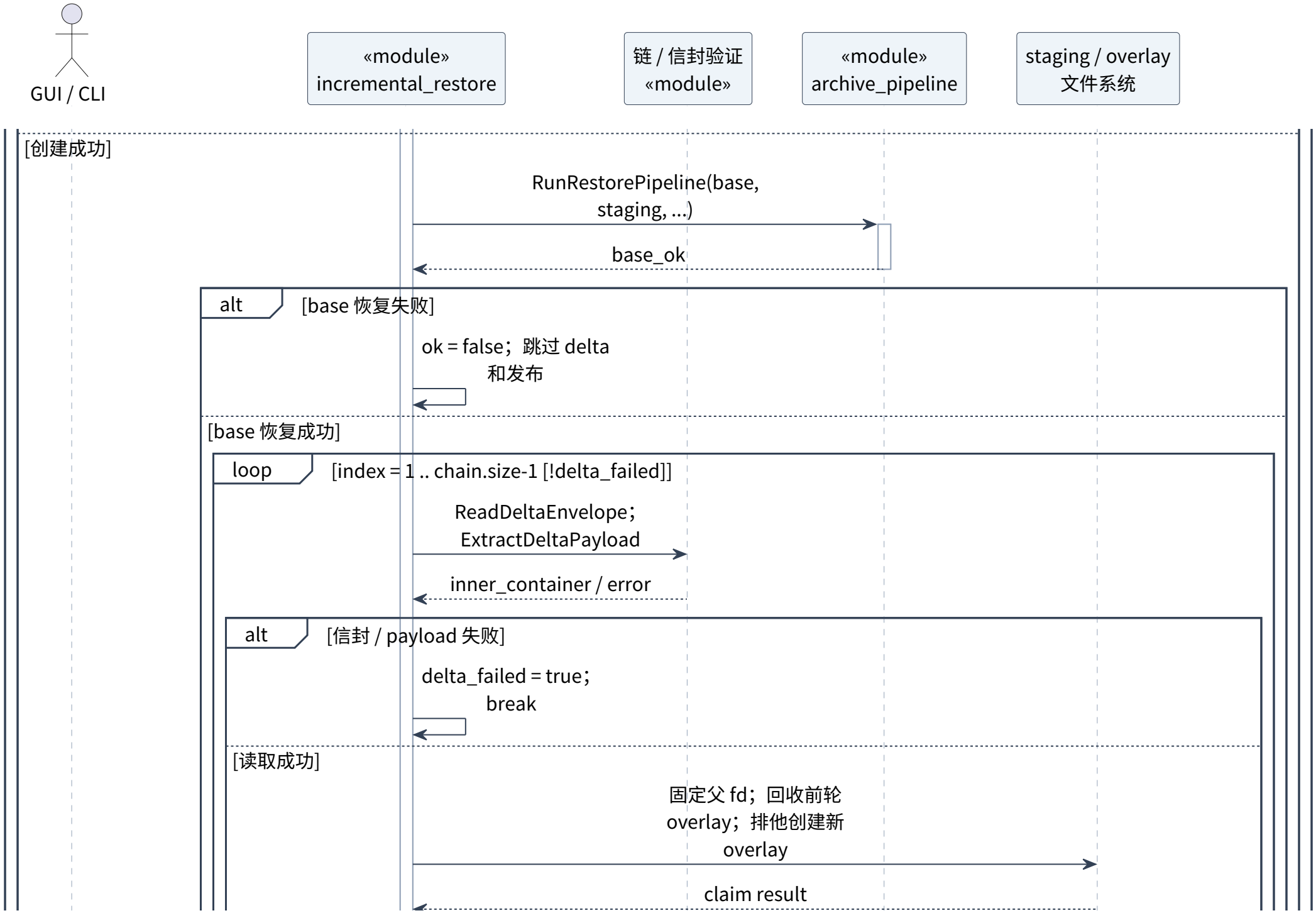
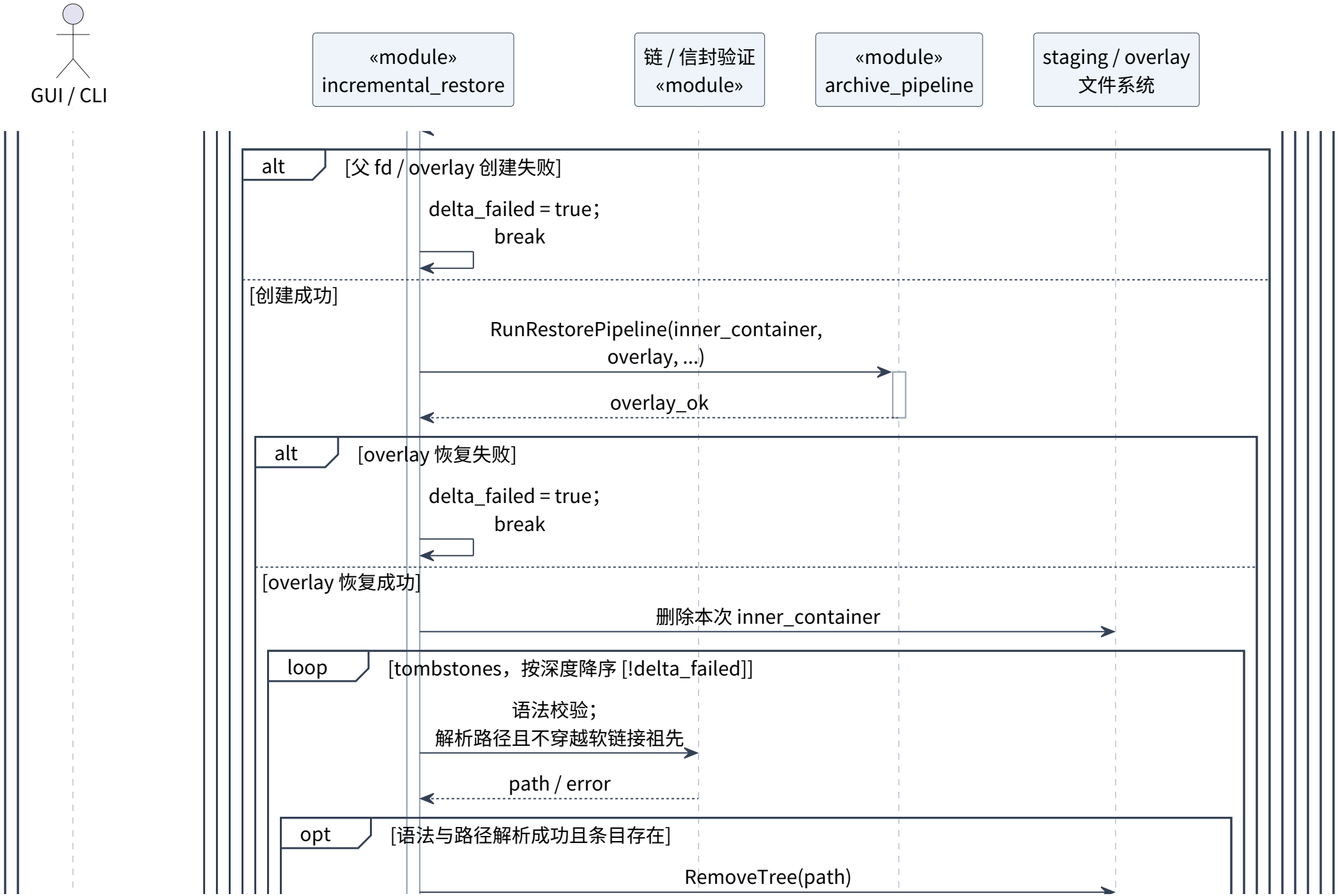
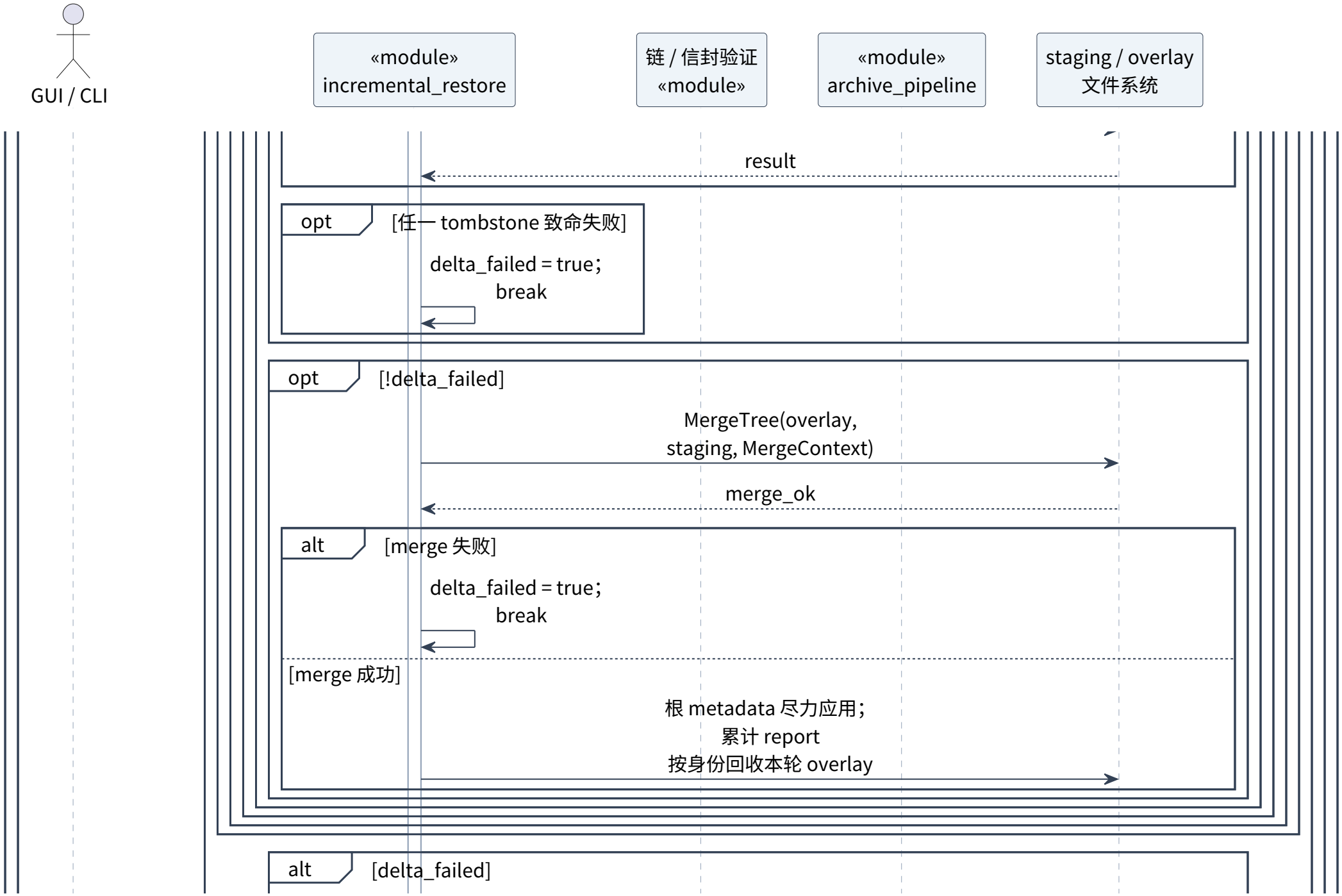
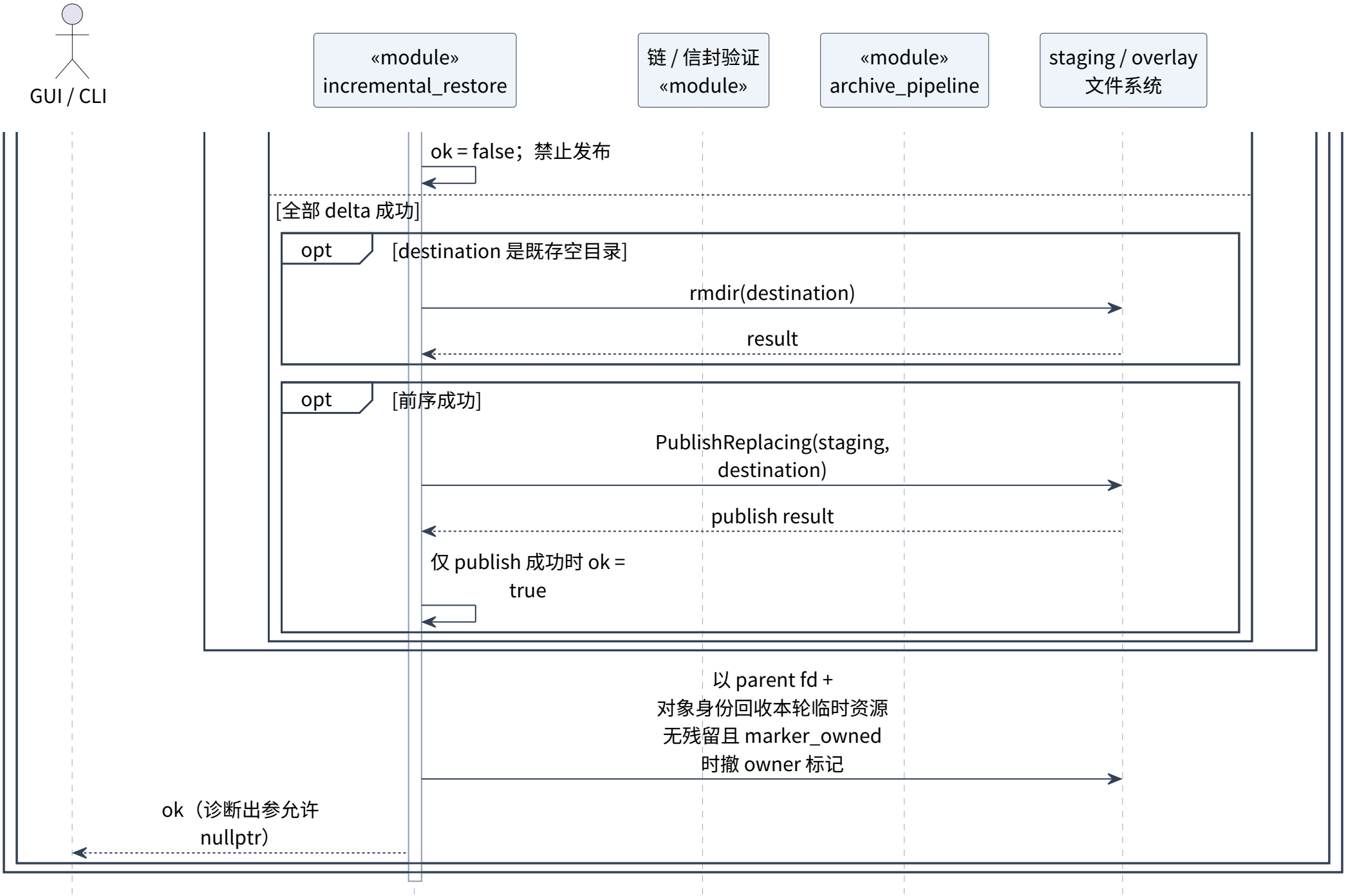

GUI / CLI
«module»
incremental_restore
链 / 信封验证
«module»
«module»
archive_pipeline
staging / overlay
文件系统
RestoreSnapshotChain(repository,
name, destination, ...)
ResolveSnapshotChain：
身份 / 父绑定 / 环 / 深度
chain / error
规范化目标；
要求不存在或为空
alt
[链或目标检查失败]
false
[前置检查通过]
建父目录并固定 parent
fd；owner 标记
排他创建 staging /
overlay；身份核验
claims / error
alt
[创建冲突或身份不可验证]
仅回收可证明归属的资源；
保留冲突对象
false；保留必要来源标记


GUI / CLI
«module»
incremental_restore
链 / 信封验证
«module»
«module»
archive_pipeline
staging / overlay
文件系统
[创建成功]
RunRestorePipeline(base,
staging, ...)
base_ok
alt
[base 恢复失败]
ok = false；跳过 delta
和发布
[base 恢复成功]
loop
[index = 1 .. chain.size-1 [!delta_failed]]
ReadDeltaEnvelope；
ExtractDeltaPayload
inner_container / error
alt
[信封 / payload 失败]
delta_failed = true；
break
[读取成功]
固定父 fd；回收前轮
overlay；排他创建新
overlay
claim result


GUI / CLI
«module»
incremental_restore
链 / 信封验证
«module»
«module»
archive_pipeline
staging / overlay
文件系统
alt
[父 fd / overlay 创建失败]
delta_failed = true;
break
[创建成功]
RunRestorePipeline(inner_container,
overlay, ...)
overlay_ok
alt
[overlay 恢复失败]
delta_failed = true;
break
[overlay 恢复成功]
删除本次 inner_container
loop
[tombstones，按深度降序 [!delta_failed]]
语法校验;
解析路径且不穿越软链接祖先
path / error
opt
[语法与路径解析成功且条目存在]
RemoveTree(path)


GUI / CLI
«module»
incremental_restore
链 / 信封验证
«module»
«module»
archive_pipeline
staging / overlay
文件系统
result
opt
[任一 tombstone 致命失败]
delta_failed = true;
break
opt
[!delta_failed]
MergeTree(overlay,
staging, MergeContext)
merge_ok
alt
[merge 失败]
delta_failed = true;
break
[merge 成功]
根 metadata 尽力应用;
累计 report
按身份回收本轮 overlay
alt
[delta_failed]


GUI / CLI
«module»
incremental_restore
链 / 信封验证
«module»
«module»
archive_pipeline
staging / overlay
文件系统
ok = false；禁止发布
[全部 delta 成功]
opt
[destination 是既存空目录]
rmdir(destination)
result
opt
[前序成功]
PublishReplacing(staging,
destination)
publish result
仅 publish 成功时 ok =
true
以 parent fd +
对象身份回收本轮临时资源
无残留且 marker_owned
时撤 owner 标记
ok（诊断出参允许
nullptr）